SensorMind: An Industrial IoT Sensor Intelligence Platform

Abstract: SensorMind is an Industrial IoT sensor intelligence platform. It shows the equipment of a site in a 3D floor view, with live sensor values and warnings beside each machine. An assistant answers questions by combining the live readings with equipment manuals, and it names the manual page it used. This report explains what the platform does, which technology it uses, and how a customer's sites, floors, machines and sensors are organised. Supermarket fridges serve as the example. All sensor data come from a built-in simulator, and the assistant was tried only against a local stand-in for the AI service.

1. What SensorMind Is

SensorMind is an Industrial IoT sensor intelligence platform. IoT means the Internet of Things: machines fitted with sensors that report their state. A user signs in, sees the machines of a site in a 3D floor view and reads the live value of each sensor. Every value is marked normal, warning or alarm, so a problem stands out.

The running example is Walnut, a fictional supermarket chain with a store in Berlin. The store has 20 fridge and freezer cabinets on two floors. Each cabinet has three sensors: cabinet temperature, compressor current and door open time. An assistant answers questions such as "Why is cabinet R-03 getting warmer?" It reads the live values and the equipment manual, and it tells the user which manual page it used. The sensor values are simulated and the manual is a fictional demo document, so no real store stands behind the example.

Why it matters. A fridge that drifts warm is easier to deal with when the warning, the readings and the likely causes are in one place.

2. Technology Used

Table 1 lists each part and what it does; Figure 1 shows how the parts connect. The browser shows the screens. The server applies the rules and talks to three outside services: a database, a search store for manuals and an AI service.

Table 1. Technology used and what each part does.

Layer	Technology	What it does
Screens	React, TypeScript	Build the pages and update them as new values arrive
3D floor view	Three.js	Draws the floor and the machines in 3D
Charts	Recharts	Draws trend lines
Styling	Tailwind CSS	Sets colours, spacing and layout
Server	ASP.NET Core 8, API controllers	Holds the rules, such as when a value is a warning, and answers requests from the screens
Customer data	PostgreSQL, Entity Framework Core	PostgreSQL is a database for customers (tenants), sites, users and a request log. Entity Framework Core is the code that reads and writes it
Manual search	Qdrant	A vector database: a database that finds text by meaning
AI models	OpenRouter	A service that gives access to AI models: one turns text into a searchable form, one writes answers
Live readings	Server-Sent Events	The server keeps pushing new values to the browser, with no page reload
Sensor values	Built-in simulator	Produces fake readings, including drifts and alarms
Local services	Docker Compose	Starts the database and Qdrant on a developer's computer
API documentation	Swagger	Lists the server's endpoints and lets a developer try them

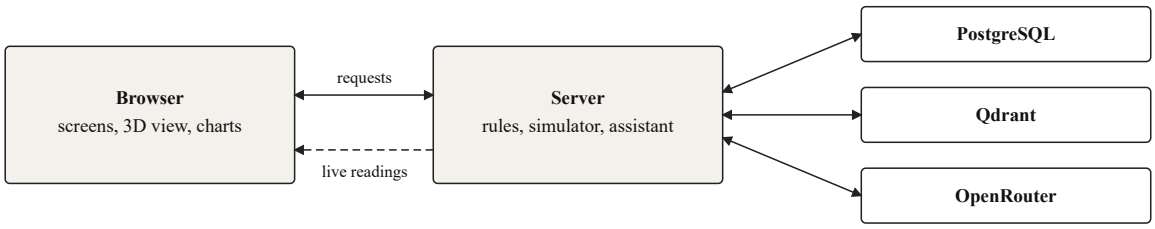


Figure 1. How the parts connect. The dashed arrow is the stream of live readings. PostgreSQL holds customer data, Qdrant holds the manual passages, and OpenRouter provides the AI models.

3. Parts of the Application

After signing in (Figure 3a) the user sees the main view (Figure 2). It has an asset tree on the left, the 3D floor in the middle, and the sensor panel and assistant on the right. To work with it, pick a floor with the tabs above the 3D view, then click a

machine (an asset) in the tree or in the 3D view. The sensor panel lists the machine's sensors (tags) with the current value, the status and a short trend line. The user can then ask the assistant a question (Figure 6).

Warnings pulse slowly in amber and alarms pulse faster in red in the 3D view, the tree, the floor tabs and the notification bell. A still image cannot show the pulse, so the figures show the colours only. The bell (Figure 3b) lists each new warning and alarm with its floor and value. The list exists only in the browser and is empty after a reload.

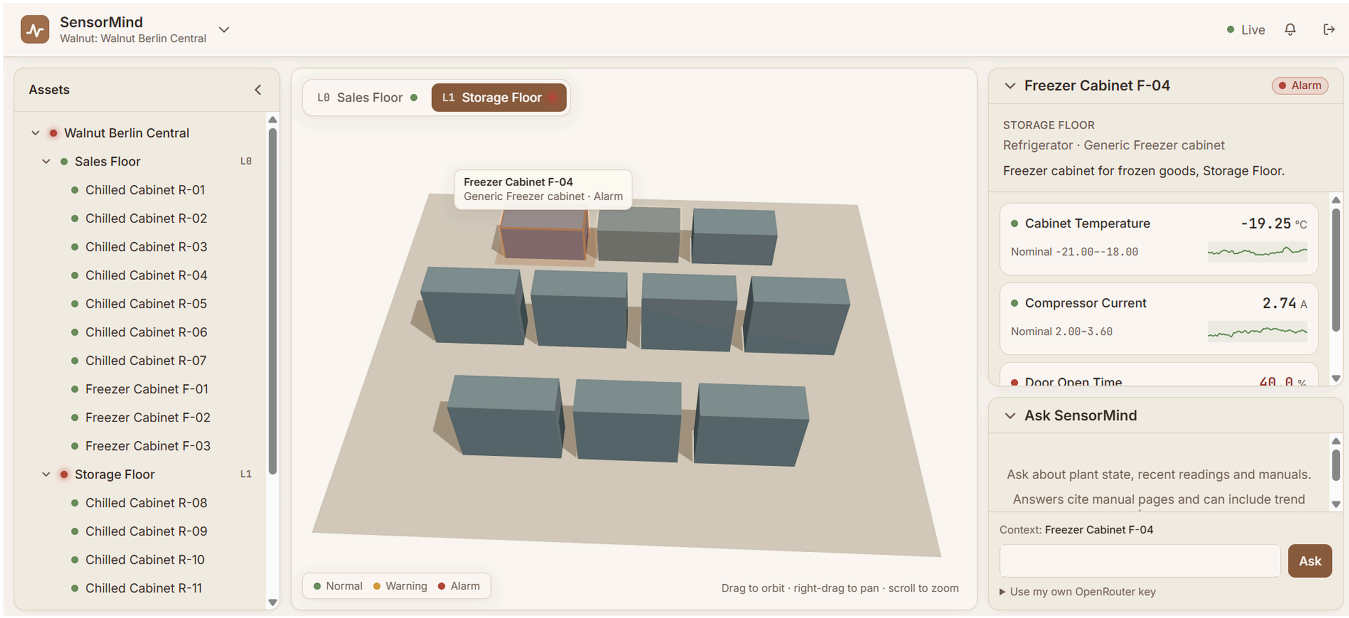
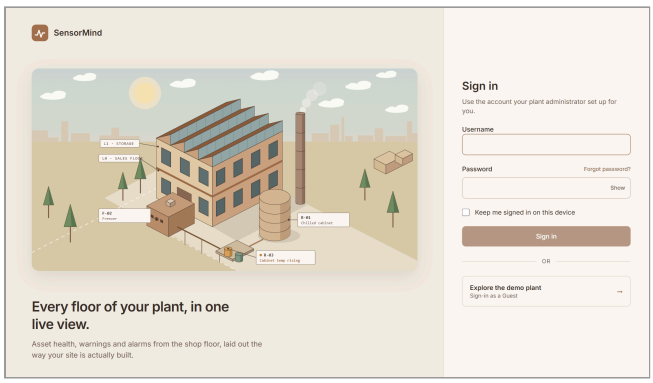
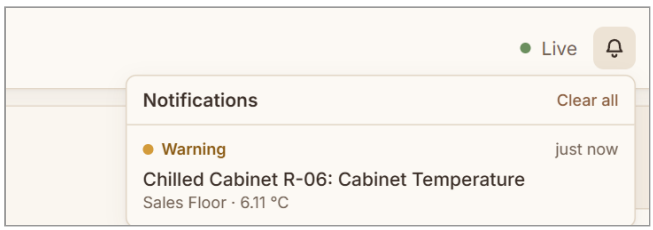


Figure 2. Main view with Freezer Cabinet F-04 selected, in alarm. This is a screenshot of the running application; the simulator produced the alarm on its own, nothing was forced. The unit is tinted red in the 3D view, and the status shows in the tree, the floor tab and the sensor panel. A still image cannot show the pulse.



(a)



(b)

Figure 3. (a) Sign-in page, with a button for a guest account. (b) Notification list with a warning that the simulator raised on its own.

4. How a Customer Is Organised

Everything is arranged in five levels (Figure 4). A tenant is a customer company, here Walnut, and it holds one or more sites. A site is a place such as a store: "Walnut Berlin Central". A site has floors, a floor holds assets (the machines), and an asset has tags (the sensor signals). Assets sit directly under a floor, with no groups in between. The tenant is stored in the database together with its sites. The floors, assets and tags of a site are written in a text file that is checked when it is loaded, and every mistake in it is reported.

Each tag has a normal range and, usually, alarm limits. A value inside the range is normal. A value outside it is a warning. A value at or beyond an alarm limit is an alarm. For Cabinet Temperature the normal range is 1 to 4 °C and the alarm limits are 7 °C and −1 °C. An asset, a floor and a site show the worst status found below them.

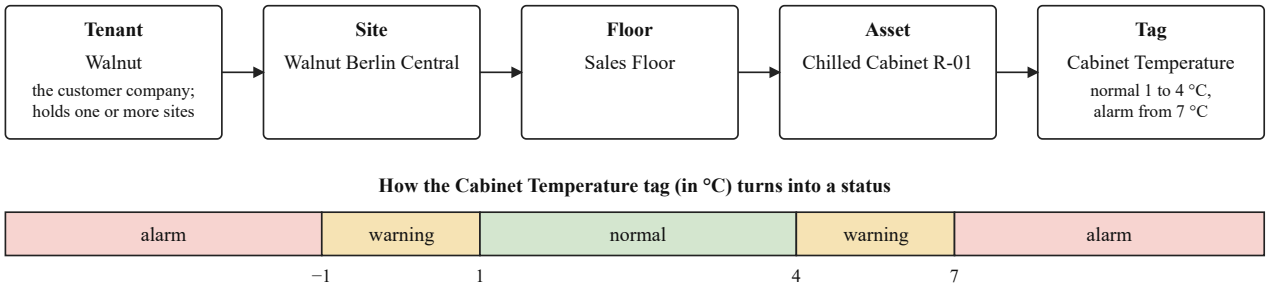


Figure 4. The five levels, with Walnut as the example. Each level holds one or more of the next. The bar below shows how one tag value maps to a status; its scale is schematic.

5. Example: What Happens at Walnut

Table 2 follows one chilled cabinet from a small drift to a likely cause.

Table 2. A cabinet warms up: the steps and what the user sees.

Step	What happens	What the user sees
1	The temperature of cabinet R-03 drifts above 4 °C.	A warning: an amber pulse on the cabinet, its floor tab and the tree, and a new item in the bell.
2	The temperature keeps rising and reaches 7 °C.	An alarm: the pulse turns red and the bell lists the change.
3	The user selects the cabinet.	Its three sensors with values, status and trend lines.
4	The user asks, “Why is cabinet R-03 getting warmer?”	The question appears in the chat panel.
5	The assistant combines the live readings with page 3 of the demo manual. That page lists causes: a door open too long or a worn door seal, blocked airflow, a dirty condenser, a defrost fault, refrigerant loss and a compressor fault. It advises checking the simplest cause first.	An answer that names likely causes and what to check, with a chip citing the manual page.

What this gives. The platform is meant to give earlier notice of a problem, one place to look instead of several tools, and causes and next steps next to the readings. No benefit was measured: all data are simulated and nothing was tested in a real store.

6. How the Assistant Answers

When a user asks a question (Figure 5), the platform does two things. It searches the manuals for passages that match the meaning of the question, and it reads the current values and recent trend of the relevant sensors. An AI model then writes the answer from these two sources and names the manual pages it used. The manuals are prepared once, when the server starts: each page is split into passages, turned into numbers that capture meaning and stored in Qdrant, and a manual already stored is skipped. If the manual search fails, the assistant still answers from sensor values.

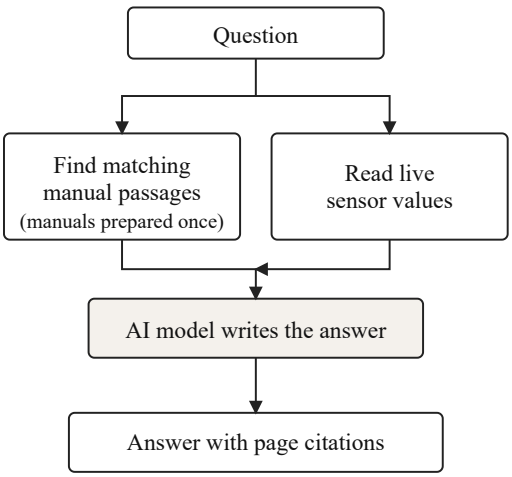


Figure 5. How a question is answered.

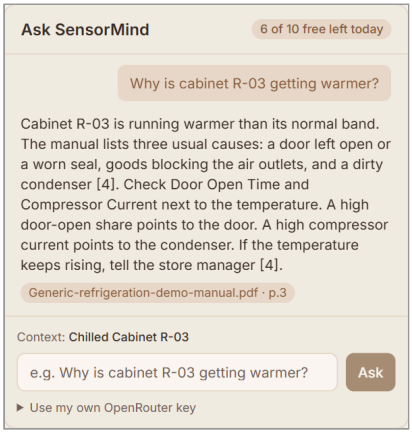


Figure 6. The assistant answering the example question. The text comes from a local stand-in for the AI service and is a fixed reply; the citation chip (page 3 of the demo manual) is the real result of the manual search.

7. Limits

The system has been checked on simulated data only. It is not connected to live sensors.

8. Conclusion

SensorMind brings sensor values, alarms, a 3D floor view and a manual-aware assistant into one platform, shown here with a fictional supermarket. The prototype runs end to end on simulated data.

Appendix A. AI-Assisted Development Workflow

Claude Code assisted with the code and the documents, and each task was handed to a separate task-executor subagent; a large part of the working time went into writing prompts, reviewing code and verifying results in the running application.

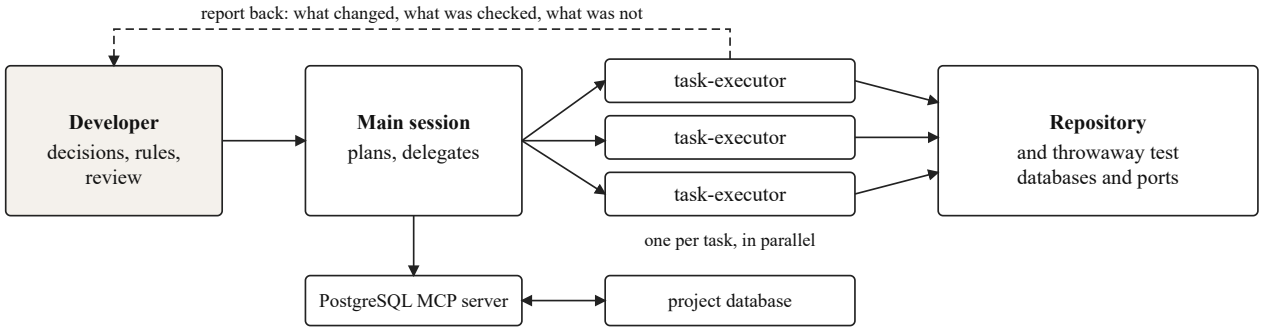


Figure A1. Development workflow: the developer gives instructions to a main session, which hands each task to its own task-executor subagent; the subagents work on the repository and on throwaway test instances and report back for review. The PostgreSQL MCP server (Model Context Protocol, a standard way to give the tool extra abilities) lets the main session inspect the project database.

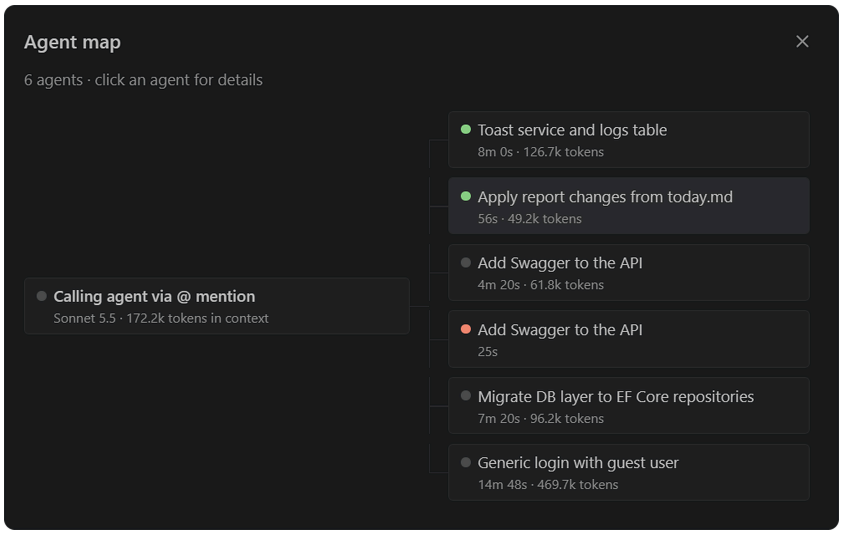


Figure A2. Agent map shown by Claude Code during development: the main session and its task-executor subagents. A screenshot; it lists six of the subagents, with the task names given by the tool.